

# dattri-LLM: A UNIFIED AND EFFICIENT LIBRARY FOR TRAINING DATA ATTRIBUTION AT LLM SCALE

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Training data attribution (TDA) estimates the contribution of individual training examples to model outputs. Most scalable TDA methods rely on per-example gradients, whose computation and use at LLM scale pose challenges in efficiency, compatibility, and extensibility. We introduce `dattri-LLM`, a TDA library that makes gradient-based attribution more practical at scale. It captures per-example gradients from any training loop that calls `backward()`, without changes to the loop or its configuration, including distributed training with DDP and FSDP and existing pipelines such as Hugging Face Transformers, TRL, and OLMo. It keeps every gradient in a compact form that a cost model expands only when doing so is cheaper, so the same captured gradients serve gradient similarity, curvature-based influence, and trajectory-based methods, at the level of examples or of individual tokens, and also support acting on gradients during training, such as online data selection. On the same hardware and workload, `dattri-LLM` runs curvature-based methods 1.3–2.5 $\times$  faster than the fastest existing library and gradient similarity at parity, scales attribution to 110B-parameter models across four GPUs, and delivers faster and more faithful attribution than existing libraries at 7B scale.

## 1 INTRODUCTION

Training data attribution (TDA) (Deng et al., 2025) estimates the contribution of individual training examples to a model’s outputs, with applications in model debugging (Koh & Liang, 2017), data selection (Xia et al., 2024), and data valuation (Ghorbani & Zou, 2019). Many scalable TDA methods are gradient-based: they compare derivatives of a query quantity with per-example training derivatives, either directly, as in TracIn (Pruthi et al., 2020), or through random projection and curvature-based transformations, as in TRAK and influence functions (Park et al., 2023; Grosse et al., 2023). Although their formulations differ, these methods share computational building blocks: obtaining per-example derivatives and applying operations such as projection, preconditioning, and inner products.

At LLM scale, these computations pose systems challenges in three respects. The first is *efficiency*. A per-example gradient usually contains billions of parameters, making its construction and storage expensive. Structured representations can reduce this cost, but their relative efficiency depends on the layer shape and the operation being performed. Supporting alternative representations through shared operations allows these choices to benefit multiple attribution methods. The second is *compatibility*. Existing libraries offer task-based attribution workflows, trainer integrations, and custom-loop gradient collectors, but additional integration work is required from the user, particularly when they are using existing training frameworks. The third is *extensibility*. Existing libraries provide reusable attribution and gradient-processing interfaces, but implementing a new method or an application that acts during training still requires coordinating gradient capture, representation, and execution timing.

We introduce `dattri-LLM`, a library for TDA at LLM scale that addresses these challenges through a unified gradient interface. Its design (§3) and evaluation (§4,§5) center on these properties:

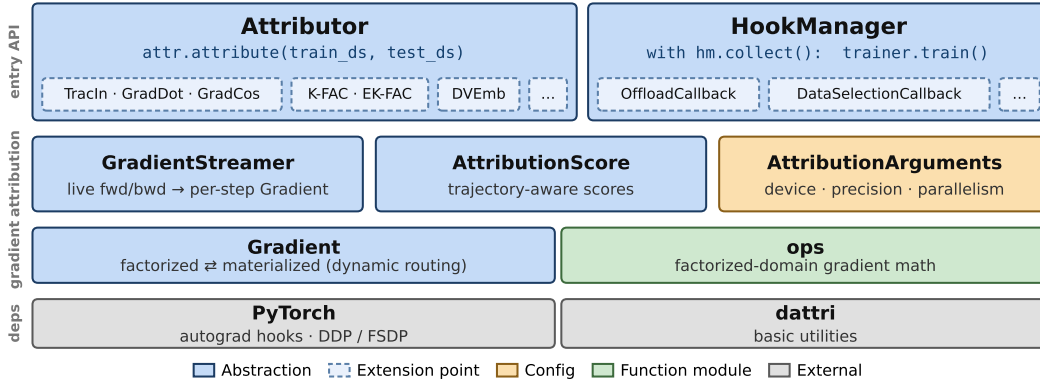


Figure 1: **Architecture overview of dattri-LLM.** Tiers are stacked bottom-up, from external dependencies (gray), through the gradient tier that represents per-example gradients and the operations on them (green), to the attribution tier holding the components shared by all methods, and the entry API at the top. Solid blue boxes are classes the user instantiates and calls, and orange marks configuration. Dashed boxes are abstract bases that double as extension points: subclassing `Attributor` adds an attribution method, and subclassing `HookManagerCallback` adds an application that acts on the capture loop. The entry API offers two ways in: `Attributor` runs attribution end-to-end, whereas `HookManager` captures gradients from any `.backward()` the user’s own training loop executes.

**Efficiency: exact factorized gradients with FLOP-aware routing.** `dattri-LLM` supports two exact representations of per-example gradients for supported layers, termed *factorized* and *materialized*. A FLOP-aware cost model selects between them per layer and per operation, reducing execution cost without introducing an additional approximation to the attribution computation. Attribution methods use a shared set of gradient operations, allowing the routing strategy to benefit their implementations through a common interface (§3.2). Across the methods and projection regimes evaluated in §B.3, `dattri-LLM` achieves 1.4–3.1× speedups over the compared libraries.

**Compatibility: non-invasive gradient capture from existing training loops.** Alongside an end-to-end attribution interface, `dattri-LLM` provides an *open-loop* interface that captures per-example gradients from existing `.backward()` execution through autograd hooks. Users enable capture by enclosing the training call in a context manager, without editing the training loop, optimizer, or trainer. We demonstrate this integration with Hugging Face `Trainer` (Wolf et al., 2020), `TRL` (von Werra et al., 2020), and `OLMo` (Groeneveld et al., 2024), and evaluate capture under DDP (Li et al., 2020) and FSDP (Zhao et al., 2023) (§3.3). Captured gradients can be cached and reused across queries and attribution methods whose gradient and checkpoint requirements are satisfied by the cache, avoiding repeated training-side gradient computation.

**Extensibility: pluggable interfaces over a shared capture path.** `dattri-LLM` exposes an `Attributor` interface for implementing gradient-based attribution methods over a uniform gradient stream and shared gradient operations. Its `HookManagerCallback` interface supports downstream applications that inspect or modify training state using captured gradients (§3.4). We demonstrate this interface by integrating a recent online data-selection method into an existing fine-tuning loop through a callback (§5).

## 2 RELATED WORK

We review TDA methods, whose shared building blocks motivates the design of `dattri-LLM`, and existing TDA libraries, against which we position this library.

**Training data attribution methods.** TDA methods estimate how individual training examples contribute to a model’s behavior, and are commonly grouped by how that contribution is com-

Table 1: Comparison of recent libraries for LLM-scale TDA. ✓/✗ indicates supported / not supported; ~ indicates partial support: Bergson integrates with a wrapped Hugging Face Trainer through callbacks that record gradients without acting on the training state. LogIX supports DDP but not FSDP. Kronfluence and Bergson provide interfaces for customizing specific steps of attribution, such as gradient collection and preconditioning, but not for defining a new end-to-end attribution method.

|                                          | dattri-LLM | LogIX | Kronfluence | Bergson |
|------------------------------------------|------------|-------|-------------|---------|
| <i>Efficiency</i>                        |            |       |             |         |
| Dimension reduction                      | ✓          | ✓     | ✗           | ✓       |
| Training-gradient reuse                  | ✓          | ✓     | ✗           | ✓       |
| <i>Compatibility</i>                     |            |       |             |         |
| Integration with existing training       | ✓          | ✗     | ✗           | ~       |
| Distributed execution                    | ✓          | ~     | ✓           | ✓       |
| <i>Extensibility</i>                     |            |       |             |         |
| Interface for new attribution methods    | ✓          | ✗     | ~           | ~       |
| Interface for training-time applications | ✓          | ✗     | ✗           | ✗       |

puted (Deng et al., 2024). *Retraining-based* methods such as Datamodels (Ilyas et al., 2022) and Simfluence (Guu et al., 2023) fit a predictor of model behavior to observations from many training runs; they are faithful to the counterfactual but their cost grows with the number of runs. *Representation-based* methods such as representer points (Yeh et al., 2018) explain predictions through learned features, at the cost of ignoring most of the network. *Gradient-based* methods, which cover most methods that scale to LLMs, compare the gradient of a query with the gradients of training examples. Influence functions (Koh & Liang, 2017) precondition this comparison by an inverse Hessian, approximated at scale by Arnoldi iteration (Schioppa et al., 2022), Kronecker-factored curvature (K-FAC and EK-FAC) (Martens & Grosse, 2015; George et al., 2018; Grosse et al., 2023), or a closed form for low-rank adapters (Kwon et al., 2024). TracIn (Pruthi et al., 2020) accumulates gradient inner products over checkpoints, and Grad-Dot and Grad-Cos (Charpiat et al., 2019) evaluate them at a single one. TRAK (Park et al., 2023), LoGRA (Choe et al., 2025), and GraSS (Hu et al., 2025) make gradients tractable through random projection or sparsification, and LESS (Xia et al., 2024) and TrackStar (Chang et al., 2025) adapt gradient similarity to data selection and pretraining attribution. In-Run Data Shapley (Wang et al., 2025a), data value embeddings (Wang et al., 2025b), and SOURCE (Bae et al., 2024) further account for the training trajectory. Despite their differences, these methods share the same building blocks of per-example gradients, projection or preconditioning, and pairwise scoring, which is the layer at which dattri-LLM is organized.

**TDA libraries.** General-purpose TDA libraries such as pyDVL (TransferLab Team, 2024), OpenDataVal (Jiang et al., 2023), and Influenciae (Picard et al., 2023) support data valuation and attribution, including game-theoretic and influence-function estimators. dattri (Deng et al., 2024) provides multiple attribution methods and a benchmark suite, with projected attributors that reach billion-parameter models. Tools developed for LLM-scale attribution include Kronfluence (Grosse et al., 2023), which implements K-FAC and EK-FAC influence functions with distributed and token-level scoring; LogIX (Choe et al., 2025), which logs projected gradients through patched HuggingFace or lightning trainers; and Bergson (Quirke et al., 2026), which collects projected or full gradients into a store and integrates with the Hugging Face Trainer. Table 1 compares these libraries with dattri-LLM along the three design principles of §3, together with graded summaries of our measurements. Across these rows, dattri-LLM covers the efficiency, compatibility, and extensibility requirements together.

### 3 DESIGN

In this section we introduce the design principles of dattri-LLM. We first give an overview of its architecture (§3.1), and then describe how each of the three properties from §1 is realized: efficiency through the choice of gradient representation (§3.2), compatibility through non-invasive gradient capture (§3.3), and extensibility through two designated extension points (§3.4). Implementation details are given in Appendix A.

### 3.1 ARCHITECTURE OVERVIEW

`dattri-LLM` is organized into four tiers (Figure 1). The bottom tier holds the dependencies: PyTorch (Paszke et al., 2019), whose autograd hooks and DDP/FSDP wrappers the library builds on, and `dattri` (Deng et al., 2024), from which it imports basic utilities such as random projectors. The *gradient* tier defines the `Gradient` object, which stores a batch of per-example gradients across selected layers, using either *factorized* or *materialized* representation that we will introduce in §3.2, together with the mathematical operations on it. The *attribution* tier holds the logic shared by all attribution methods: the `GradientStreamer` that produces per-step gradients from a model, the `AttributionScore` result container, the `AttributionArguments` configuration, and the callbacks that act on the capture loop. The top tier is the entry API, which offers two ways in. `Attributor` is the *closed-loop* interface: given a model, training set, and query set, it directly returns scores for TracIn (Pruthi et al., 2020), gradient dot product and cosine, K-FAC and EK-FAC influence functions (Martens & Grosse, 2015; George et al., 2018; Grosse et al., 2023), and data-value embeddings (Wang et al., 2025b). `HookManager` is the *open-loop* interface: it captures per-example gradients from any `.backward()` executed inside a `with` block, so attribution can be attached to a training loop the user already owns (§3.3).

### 3.2 EFFICIENCY: EXACT FACTORIZED GRADIENTS WITH FLOP-AWARE ROUTING

The cost of TDA at LLM scale is dominated by computing and comparing per-example gradients. `dattri-LLM`’s efficiency relies on the following observation: the same per-example gradient can be represented in two exact forms, the operations of which are cheap in different regimes, so a library should decide the suitable one to use under different situations.

**Two representations of gradients.** Consider a linear layer  $z = Wa$  with  $W \in \mathbb{R}^{N_o \times N_i}$ , per-example input  $a \in \mathbb{R}^{N_i \times T}$  over  $T$  token positions, and output gradient  $g \in \mathbb{R}^{N_o \times T}$  with respect to the per-example loss. By the chain rule the per-example weight gradient is  $DW = g a^\top$ , i.e.,

$$\text{vec}(DW) = \sum_{t=1}^T a_t \otimes g_t, \quad (1)$$

where  $a_t$  and  $g_t$  are the columns at position  $t$  and  $\otimes$  denotes outer product. We call the dense matrix  $DW$  the *materialized* representation and the pair  $(a, g)$  the *factorized* representation. A materialized gradient can always be recovered from its factorized representation. The factorized form stores  $T(N_i + N_o)$  values against  $N_i N_o$  of the materialized form, and is smaller whenever  $T < N_i N_o / (N_i + N_o)$ , which holds for the wide layers that make up most of an LLM. Similarly, convolutional, embedding layers admit unfolding maps into the same form (Appendix A.1), making these two representations applicable to nearly all layer types.

**Postprocessing on factors.** Optional transforms can be applied to the factorized representation before scoring, including *dimension reduction* and *optimizer preconditioning*. Dimension reduction is available in factorized and materialized form. *Factor projection* applies random matrices  $P_a \in \mathbb{R}^{k_a \times N_i}$  and  $P_g \in \mathbb{R}^{k_g \times N_o}$  to the two sides,

$$\tilde{a}_t = P_a a_t, \quad \tilde{g}_t = P_g g_t, \quad \sum_{t=1}^T \tilde{a}_t \otimes \tilde{g}_t = (P_a \otimes P_g) \text{vec}(DW), \quad (2)$$

which preserves Eq. (1) and hence every subsequent operation (Choe et al., 2025; Hu et al., 2025). *Materialized projection* instead applies a single matrix to the flattened gradient,  $\tilde{\gamma} = P \text{vec}(DW)$ , as in TRAK (Park et al., 2023), and serves layers whose gradient is not an outer product. In either form, the random map may be Gaussian (Johnson & Lindenstrauss, 1984), Rademacher (Achlioptas, 2003), a sparse Johnson–Lindenstrauss transform (Kane & Nelson, 2014), the sparsified projection of GraSS (Hu et al., 2025), or a random mask that retains a subset of the entries of  $DW$ , gathered directly from the factors (Appendix A.2). *Optimizer preconditioning* replaces the raw per-example gradient by the update it induces under the training optimizer, an elementwise function of the gradient and the optimizer’s moments at that step. For Adam (Kingma & Ba, 2015),

$$\Gamma(z) = \frac{\hat{m}_z}{\sqrt{\hat{v}_z + \epsilon}}, \quad m_z = \beta_1 m + (1 - \beta_1) g_z, \quad v_z = \beta_2 v + (1 - \beta_2) g_z^2, \quad (3)$$

where  $(m, v)$  are the optimizer’s first and second moments and  $\hat{m}_z, \hat{v}_z$  their bias-corrected values (Appendix A.3). This transform enables optimizer-aware methods such as LESS (Xia et al., 2024) and AdamW-influence (Deng et al., 2026).

**Factorized inner product.** For two examples with  $T_1$  and  $T_2$  positions, the identity  $\langle u \otimes v, p \otimes q \rangle = \langle u, p \rangle \langle v, q \rangle$  gives

$$\left\langle DW^{(1)}, DW^{(2)} \right\rangle_F = \sum_{t=1}^{T_1} \sum_{s=1}^{T_2} \left\langle a_t^{(1)}, a_s^{(2)} \right\rangle \left\langle g_t^{(1)}, g_s^{(2)} \right\rangle, \quad (4)$$

so exact inner products between factorized gradients can be performed without recovering the gradient. The same identity underlies the K-FAC and EK-FAC kernels, which precondition one side first. Because the factorized form retains the token axis, it also yields attribution at token granularity.

**FLOP-aware routing.** Eq. equation 4 compares all pairs of positions and is quadratic in sequence length, whereas materializing first and taking dense inner products pays a one-time construction cost amortized over comparisons. For a cross-Gram matrix between batches of  $B_1$  and  $B_2$  examples the two routes cost  $O(B_1 B_2 T^2 (N_i + N_o))$  and  $O((B_1 + B_2) T N_i N_o + B_1 B_2 N_i N_o)$ , respectively. Neither dominates: short sequences and wide layers favor factors, long sequences and large comparison batches favor materialization, and factor projection shrinks  $N_i, N_o$  to  $k_a, k_g$  and flips the balance again. `dattri-LLM` therefore evaluates these costs per layer and per operation and takes the cheaper route (Appendix A.4), which effectively reduces the cost. An ablation in Appendix B.4 demonstrates the effectiveness of this FLOP-aware routing strategy.

### 3.3 COMPATIBILITY: NON-INVASIVE GRADIENT CAPTURE FROM ARBITRARY TRAINING LOOPS

In `dattri-LLM`, attribution can be integrated into an existing pipeline without modifying its training loop. Specifically, the *open-loop* interface `HookManager` registers autograd hooks on selected layers and captures the two factors in Eq. equation 1 during each backward pass triggered by `.backward()`. This mechanism applies uniformly to pretraining, supervised fine-tuning, and reinforcement-learning loops. For example, unmodified Hugging Face `Trainer` (Wolf et al., 2020), `TRL` (von Werra et al., 2020), and `OLMo` (Groeneveld et al., 2024) pipelines can be instrumented simply by wrapping `trainer.train()` with `with hm.collect()`. The design also naturally accommodates distributed training frameworks such as `DDP` (Li et al., 2020) and `FSDP` (Zhao et al., 2023). Captured gradients may optionally be cached, allowing the same attribution code to score them either online or later from a store persisted during an ordinary training run. Appendix A.5 details the mechanisms underlying these capabilities.

### 3.4 EXTENSIBILITY: PLUGGABLE INTERFACES OVER A SINGLE CAPTURE PATH

`dattri-LLM` provides two extension points. The `Attributor` abstraction supports the implementation of new gradient-based attribution methods. `GradientStreamer` exposes a uniform, well-defined interface to collect per-example gradients. The gradient tier provides the operations required by attribution methods, including inner products, norms, projections, and Kronecker-factored statistics, with each operation routed as described in §3.2. A new method can therefore be implemented as a single subclass that consumes these modules and produces the desired attribution scores. The built-in `TracIn`, `K-FAC`, `EK-FAC`, and `data-value-embedding` attributors are implemented through this same interface.

The second extension point, `HookManagerCallback`, supports downstream applications that operate on gradients during training. A callback receives the capture events emitted at each step and may inspect or modify the training state. Built-in examples include an offloading callback that persists gradients to disk, and a data-selection callback that scores the current batch against a target gradient, and then removes the contributions of low-scoring examples from the parameter gradients before the optimizer step. As demonstrated in §5, this interface reduces the integration of a recent online data-selection method to only a few lines of user code.

## 4 ATTRIBUTING LLMs AT SCALE WITH HIGH EFFICIENCY

In this section, we formally evaluate the efficiency of `datttri-LLM` through throughput and memory scaling, and demonstrate the favorable fidelity–cost trade-offs provided by gradient-based attribution methods as model size increases. Supplementary comparisons at a fixed model size and batch size appear in Appendix B.3.

### 4.1 SCALING ATTRIBUTION WITH MODEL SIZE

**Setup.** We compare `datttri-LLM`, LogIX (Choe et al., 2025), Bergson (Quirke et al., 2026), and Kronfluence (Grosse et al., 2023) on GRADDOT, K-FAC, and EK-FAC, using Qwen models from 0.5B to 110B parameters and 512-token WikiText-103 sequences (Merity et al., 2017) in `bf16`. We use factor projection with  $k_a = k_g = 64$  where supported. Throughput is measured on four H200 GPUs (141 GB each), using each library’s supported distributed mode and the largest per-device batch that completes within the benchmark limits. Each run attributes 512 training sequences, and we use the largest power-of-two batch size that fits in GPU memory. Detailed configurations and timing protocols are reported in Appendix B.1.

**Results.** Figure 2 (top) shows that `datttri-LLM` consistently improves attribution throughput across model scales. For GRADDOT, it achieves  $2.7\text{--}3.4\times$  the throughput of Kronfluence and  $1.4\text{--}8.9\times$  that of Bergson across their respective completed scales. The advantage also persists when compression and batch size are matched: for K-FAC and EK-FAC, `datttri-LLM` achieves on average  $3.8\times$  the throughput of LogIX from 0.5B to 32B, and up to  $6.8\times$  at 0.5B. The larger gains over Bergson and Kronfluence on curvature-based methods additionally reflect the cost of their full-dimensional curvature factors. Beyond improving throughput, `datttri-LLM` extends the range of feasible workloads: it supports all three methods through 110B, whereas the evaluated alternatives either exceed the resource budget or lack the sharded execution needed at larger scales.

To understand the memory requirements underlying these scaling results, we also evaluate attribution at batch size one, minimizing batch-dependent memory costs and highlighting the footprint of the model and attribution computation. Figure 2 (bottom) illustrates how the gradient-processing strategy affects scalability. The benefit is most pronounced for curvature-based attribution: at 3B, `datttri-LLM` uses approximately  $4.8\text{--}5.7\times$  less memory than Kronfluence and  $7.0\times$  less than Bergson for K-FAC and EK-FAC. Reducing the attribution-specific memory footprint leaves more capacity for the model and enables larger workloads within the available device memory. Together with sharded execution, this allows `datttri-LLM` to retain support for curvature-based methods at scales where the full-dimensional baseline configurations become infeasible.

### 4.2 ATTRIBUTION FIDELITY AND COMPUTATIONAL COST

The gradient-based methods supported by `datttri-LLM` approximate the effect of training-data perturbations through first-order sensitivity. We therefore complement the efficiency evaluation with counterfactual retraining, examining whether these approximations provide useful attribution at a practical computational cost.

**Setup.** We evaluate the fidelity and computational cost of attribution methods on training trajectories of GPT-2 (124M), Qwen2.5-1.5B, and OLMo-3-7B (Groeneveld et al., 2024). For each model, we construct 512 training sequences from WikiText-2 using the model’s tokenizer, with 128 tokens per sequence, and continually pretrain all parameters for one epoch using AdamW with batch size 32. OLMo-3-7B is trained through the unmodified OLMo-core trainer, with `datttri-LLM` capturing gradients by wrapping its `fit()` call. We assess fidelity against trajectory-specific leave-one-out retraining (TSLOO) (Wang et al., 2025b). For each of 50 randomly selected training sequences, we replay training with that sequence removed and measure the resulting loss changes on 64 held-out validation sequences. For each validation query, we compute the Spearman correlation between attribution scores and these loss changes across the 50 training sequences, then average over queries. Training and evaluation use `fp32` with `TF32` and dropout disabled to resolve the small counterfactual loss changes.

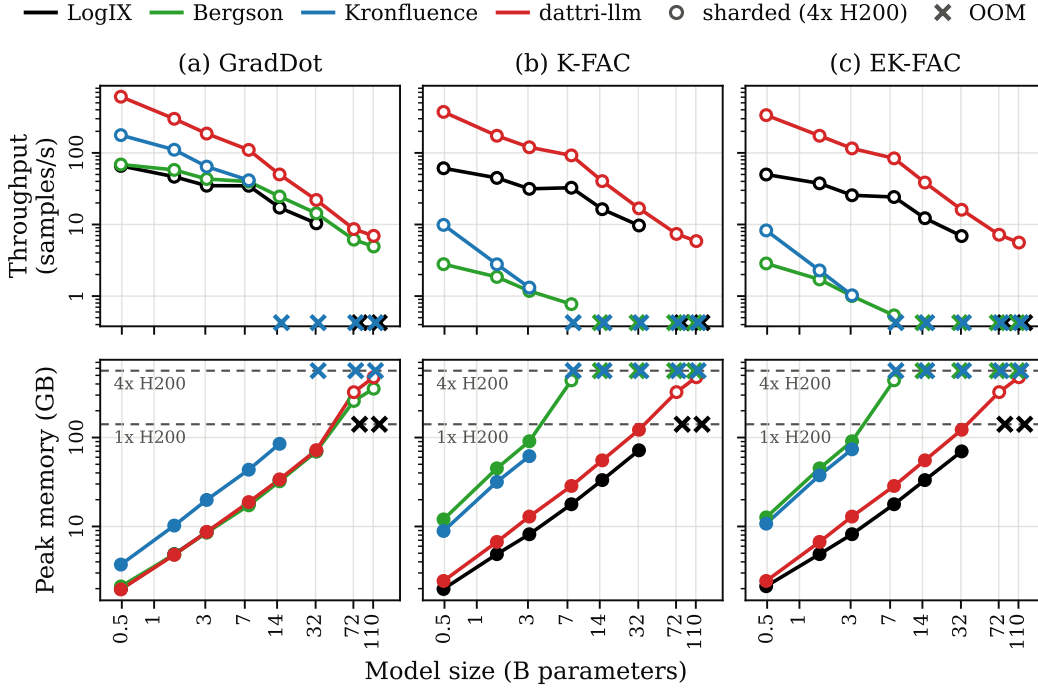


Figure 2: **Throughput and memory scaling on the Qwen family.** **Top:** throughput on four H200s at each configuration’s largest batch that completes within the benchmark limits. Crosses indicate configurations reported as out of memory. **Bottom:** GPU memory at batch size one. Filled markers denote single-H200 runs; hollow markers denote four-H200 sharded runs, with reported aggregate memory estimates. Dashed lines indicate aggregate device capacities, and crosses mark the total capacity at which an out-of-memory failure occurs.

We compare AdamW-influence (Deng et al., 2026) and EK-FAC implemented in `dattri-LLM` with Bergson’s MAGIC (Ilyas & Engstrom, 2025), SOURCE (Bae et al., 2024), TrackStar (Chang et al., 2025), and EK-FAC (Grosse et al., 2023). For AdamW-influence, we evaluate full-dimensional gradients and reduced representations that average scores from ten disjoint random masks of  $k \in \{512, 8192\}$  gradient coordinates per layer. For EK-FAC, we evaluate full-dimensional factors and projected factors with  $k_a = k_g = 64$ . Each execution receives one B200, 256 GB of host memory, and 1 TB of local disk. Attribution throughput measures the number of queries scored per second, excluding trajectory training, model and data loading, and TSLOO reference construction. Appendix B.2 provides complete settings, results, and resource limits.

**Results.** Figure 3 shows that `dattri-LLM` offers a favorable fidelity–cost trade-off across all three models, with its configurations forming the empirical Pareto frontier among completed runs in two of the three panels. On GPT-2, full-dimensional AdamW-influence achieves a correlation of 0.845, compared with MAGIC’s 0.833, while requiring less time. Masking and projection provide further control over this trade-off: for AdamW-influence, increasing the mask size  $k$  from 512 to 8192 improves correlation across all three models at the cost of approximately 2–3 $\times$  longer attribution time. Beyond attribution efficiency, `dattri-LLM`’s non-invasive gradient capture enables end-to-end attribution for OLMo-3-7B trained with the unmodified OLMo-core trainer, demonstrating compatibility with an existing training pipeline at larger scale.

## 5 INTEGRATING AND EXTENDING ATTRIBUTION WORKFLOWS

We further demonstrate how the interfaces of §3.4 make a downstream application easy to build, highlighting the compatibility and extensibility of `dattri-LLM`. We take the most recent attribution-based data selection technique for post-training (Hu et al., 2026) and deploy it inside a

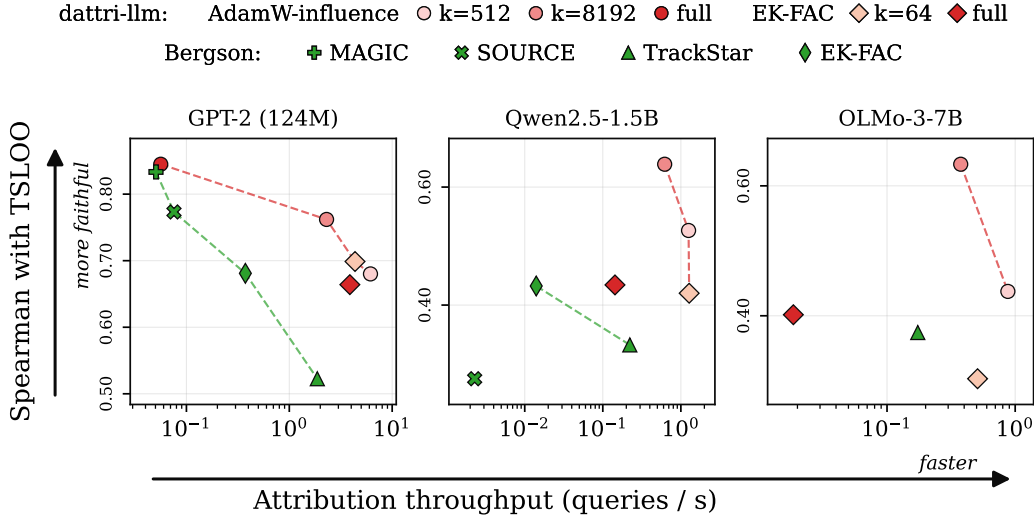


Figure 3: **Attribution fidelity versus throughput.** Higher and farther right is better. Fidelity is measured as the mean query-wise Spearman correlation with TSLOO over 64 validation queries. Throughput is 64 divided by the attribution time in seconds on one B200. Dashed lines show the empirical Pareto frontier for each library. Configurations exceeding the resource budget or not run are omitted.

minimal fine-tuning loop. Specifically, at each training step we compute GRADDOT scores between the samples in the current batch and a held-out validation set, and drop the lowest-scoring samples from the gradient update.

**Setup.** Following Hu et al. (2026), we fine-tune Llama-3.2-1B (Grattafiori et al., 2024) on Alpaca (Taori et al., 2023) with SAMSum (Gliwa et al., 2019) as the target task, dropping the bottom half of each batch at every step, and compare three variants: *Full-Training*, which trains without data selection; *Global Subset*, which ranks samples by GRADDOT scores aggregated over all layers; and *Layer-Wise Subset*, which ranks samples independently for each layer. All variants are run under both full-parameter and LoRA (Hu et al., 2022) fine-tuning. Hyperparameters are reported in Appendix B.7. Figure 4 shows the core integration, which reduces to three steps. First, the selection policy is expressed as a `DataSelectionCallback` (§3.4), which specifies the target loader and the selection rule. Second, the callback is attached to a `HookManager`, which captures per-sample gradients during the forward and backward passes at almost no additional cost. Third, the user’s training loop is enclosed in `hm.collect()`, with the loop itself unchanged. Between the backward pass and the optimizer step, the callback scores the batch and removes the dropped samples’ contributions from the parameter gradients. The same procedure applies to LoRA fine-tuning, which differs from full-parameter fine-tuning only in a layer selector that restricts the hooks to the adapters.

```
from dattri_llm import DataSelectionCallback,
HookManager
# 1. Define the selection policy
select = DataSelectionCallback(
    model, target="val_loader",
    val_loader=target_loader, val_loss_fn=loss_fn,
    selection_kwargs={"threshold": 0.5, "
    threshold_mode": "bottom_fraction"})
# 2. Define the capture hook
hm = HookManager(model, callbacks=[select])
# 3. Perform data selection
with hm.collect():
    for batch in train_loader:
        loss_fn(model, batch).backward()
        optimizer.step(); optimizer.zero_grad()
```

Figure 4: Online data selection with dattri-LLM.

**Results.** Figure 5 shows the training curves. Both selection variants outperform Full-Training in both regimes, and Layer-Wise Subset performs best, reaching a final target perplexity of 5.11 vs. 6.31 under full-parameter fine-tuning and 4.87 vs. 6.31 under LoRA, with gaps far exceeding the seed variance. Full-Training overfits the off-target source data and its perplexity drifts upward,

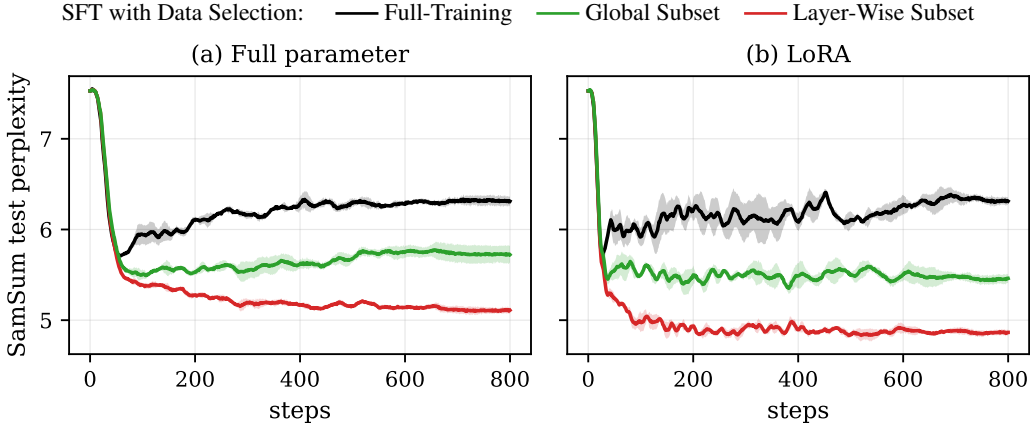


Figure 5: Attribution-guided data selection under (a) full-parameter and (b) LoRA fine-tuning (SAMSum test perplexity, lower is better; mean over 3 seeds, band  $\pm 1$  std). Both selection variants improve on training with all data, with per-layer selection best in both regimes.

whereas selection acts as a regularizer that steers each update toward target-aligned examples, and the finer-grained per-layer selection regularizes further. The result reproduces the finding of Hu et al. (2026) using the integration of Figure 4 in place of a dedicated training pipeline. Notably, the `dattri-LLM` implementation incurs negligible runtime overhead compared with the original implementation by Hu et al. (2026). We report runtime measurements in Appendix B.7.

## 6 CONCLUSION

We presented `dattri-LLM`, a library for training data attribution at LLM scale. Its design rests on three properties. For efficiency, `dattri-LLM` represents per-example gradients exactly in either factorized or materialized form and routes each operation to the cheaper form with a FLOP-aware cost model, so that a single set of kernels serves every attribution method and every supported layer type without changing the resulting scores. For compatibility, gradient capture is non-invasive: it attaches to an existing training loop through autograd hooks, runs on unmodified training pipelines, and scales out under DDP and FSDP. For extensibility, `dattri-LLM` exposes a callback interface over its capture path, so downstream applications such as online data selection are built by composing callbacks rather than by modifying the core. Empirically, `dattri-LLM` is  $1.4\text{--}3.1\times$  faster than existing libraries across methods and projection regimes. We believe `dattri-LLM` can lower the systems barrier to studying how training data shapes large language models.

## AI USE STATEMENT

In this work, we used generative AI tools for implementing methods. We have not used generative AI tools to help develop theoretical models or conceptual frameworks, formulate mathematical claims, provide critical ingredients for proving mathematical claims, assist in the writing of proofs, propose or refine hypotheses, design or provide feedback on research methodology or experiments, assist with translation, clean and reformat dataset, or interpret results, and Generate synthetic data sets and support qualitative and thematic data analysis are not applicable to this work. Additionally, we used generative AI tools to create or modify scientific figures or images, draft parts of a research paper, edit a research paper to improve readability, and format references. We have reviewed all AI-assisted work. The authors reviewed AI-assisted implementations, figures, text, and references for correctness and consistency with the methods and experimental results. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

## REPRODUCIBILITY STATEMENT

We document the library design and experimental procedures to support reproducibility. All experimental details are included in Appendix B. Appendix B.1 specifies the detailed setup for the throughput measurement experiment, including the hardware, model checkpoints, workloads, distributed configurations, batch-size selection, and timing and memory measurement protocols. Appendix B.2 details the fidelity evaluation, including training settings, counterfactual retraining, numerical controls, and evaluation metrics. Appendices B.3, B.4, and B.5 provide the settings for matched-batch comparisons, routing ablations, and token-level attribution examples. Appendix B.7 documents the datasets, hyperparameters, selection procedure, and evaluation protocol for online data selection, with results reported over three random seeds. We will officially release the code upon acceptance of the paper.

## REFERENCES

- Dimitris Achlioptas. Database-friendly random projections: Johnson–Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003.
- Juhan Bae, Wu Lin, Jonathan Lorraine, and Roger Grosse. Training data attribution via approximate unrolling. In *Advances in Neural Information Processing Systems*, volume 37, pp. 66647–66686, 2024.
- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023.
- Tyler A. Chang, Dheeraj Rajagopal, Tolga Bolukbasi, Lucas Dixon, and Ian Tenney. Scalable influence and fact tracing for large language model pretraining. In *International Conference on Learning Representations*, 2025.
- Guillaume Charpiat, Nicolas Girard, Loris Felardos, and Yuliya Tarabalka. Input similarity from the neural network perspective. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Sang Keun Choe, Hwijeen Ahn, Juhan Bae, Kewen Zhao, Youngseog Chung, Adithya Pratapa, Willie Neiswanger, Emma Strubell, Teruko Mitamura, Jeff Schneider, Eduard Hovy, Roger Grosse, and Eric P. Xing. What is your data worth to GPT? LLM-scale data valuation with influence functions. In *Advances in Neural Information Processing Systems*, volume 38, pp. 145944–145985, 2025.
- Junwei Deng, Ting-Wei Li, Shiyuan Zhang, Shixuan Liu, Yijun Pan, Hao Huang, Xinhe Wang, Pingbang Hu, Xingjian Zhang, and Jiaqi W. Ma. dattri: A library for efficient data attribution. In *Advances in Neural Information Processing Systems*, volume 37, pp. 136763–136781, 2024.
- Junwei Deng, Yuzheng Hu, Pingbang Hu, Ting-wei Li, Shixuan Liu, Jiachen T. Wang, Dan Ley, Qirun Dai, Benhao Huang, Jin Huang, Cathy Jiao, Hoang Anh Just, Yijun Pan, Jingyan Shen, Yiwen Tu, Weiyi Wang, Xinhe Wang, Shichang Zhang, Shiyuan Zhang, Ruoxi Jia, Himabindu Lakkaraju, Hao Peng, Weijing Tang, Chenyan Xiong, Jieyu Zhao, Hanghang Tong, Han Zhao, and Jiaqi W. Ma. A survey of data attribution: Methods, applications, and evaluation in the era of generative AI. *SSRN preprint*, 2025. doi: 10.2139/ssrn.5451054. URL <https://ssrn.com/abstract=5451054>.
- Junwei Deng, Pingbang Hu, Suliang Jin, Hao Lu, Jiachen T. Wang, Shichang Zhang, and Jiaqi W. Ma. How faithful is trajectory-based data attribution? error sources, remedies, and practical guidelines, 2026. URL <https://arxiv.org/abs/2605.18814>.
- Thomas George, César Laurent, Xavier Bouthillier, Nicolas Ballas, and Pascal Vincent. Fast approximate natural gradient descent in a Kronecker factored eigenbasis. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

- Amirata Ghorbani and James Zou. Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2242–2251. PMLR, 2019.
- Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, pp. 70–79. Association for Computational Linguistics, 2019.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Dirk Groeneveld, Iz Beltagy, Evan Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, Shane Arora, David Atkinson, Russell Authur, Khyathi Chandu, Arman Cohan, Jennifer Dumas, Yanai Elazar, Yuling Gu, Jack Hessel, Tushar Khot, William Merrill, Jacob Morrison, Niklas Muennighoff, Aakanksha Naik, Crystal Nam, Matthew Peters, Valentina Pyatkin, Abhilasha Ravichander, Dustin Schwenk, Saurabh Shah, William Smith, Emma Strubell, Nishant Subramani, Mitchell Wortsman, Pradeep Dasigi, Nathan Lambert, Kyle Richardson, Luke Zettlemoyer, Jesse Dodge, Kyle Lo, Luca Soldaini, Noah Smith, and Hannaneh Hajishirzi. OLMo: Accelerating the science of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15789–15809. Association for Computational Linguistics, 2024.
- Roger Grosse, Juhan Bae, Cem Anil, Nelson Elhage, Alex Tamkin, Amirhossein Tajdini, Benoit Steiner, Dustin Li, Esin Durmus, Ethan Perez, Evan Hubinger, Kamilė Lukošūtė, Karina Nguyen, Nicholas Joseph, Sam McCandlish, Jared Kaplan, and Samuel R. Bowman. Studying large language model generalization with influence functions. *arXiv preprint arXiv:2308.03296*, 2023.
- Kelvin Guu, Albert Webson, Ellie Pavlick, Lucas Dixon, Ian Tenney, and Tolga Bolukbasi. Simfluence: Modeling the influence of individual training examples by simulating training runs. *arXiv preprint arXiv:2303.08114*, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Pingbang Hu, Joseph Melkonian, Weijing Tang, Han Zhao, and Jiaqi W. Ma. GraSS: Scalable data attribution with gradient sparsification and sparse projection. In *Advances in Neural Information Processing Systems*, volume 38, pp. 43880–43904, 2025.
- Pingbang Hu, Xueshen Liu, Z. Morley Mao, and Jiaqi W. Ma. Dr. post-training: A data regularization perspective on LLM post-training. *arXiv preprint arXiv:2605.07063*, 2026.
- Andrew Ilyas and Logan Engstrom. Magic: Near-optimal data attribution for deep learning. *arXiv preprint arXiv:2504.16430*, 2025.
- Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. Data-models: Understanding predictions with data and data with predictions. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 9525–9587. PMLR, 2022.
- Kevin Fu Jiang, Weixin Liang, James Zou, and Yongchan Kwon. OpenDataVal: A unified benchmark for data valuation. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- Daniel M. Kane and Jelani Nelson. Sparser Johnson–Lindenstrauss transforms. *Journal of the ACM*, 61(1):4:1–4:23, 2014.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

- Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1885–1894. PMLR, 2017.
- Yongchan Kwon, Eric Wu, Kevin Wu, and James Zou. DataInf: Efficiently estimating data influence in LoRA-tuned LLMs and diffusion models. In *International Conference on Learning Representations*, volume 2024, pp. 21921–21942, 2024.
- Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13(12):3005–3018, 2020. doi: 10.14778/3415478.3415530.
- James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 2408–2417. PMLR, 2015.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.
- Sung Min Park, Kristian Georgiev, Andrew Ilyas, Guillaume Leclerc, and Aleksander Madry. TRAK: Attributing model behavior at scale. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 27074–27113. PMLR, 2023.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Agustin Martin Picard, Lucas Hervier, Thomas Fel, and David Vigouroux. Influenciæ: A library for tracing the influence back to the data-points. <https://hal.science/hal-04284178>, 2023.
- Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. Estimating training data influence by tracing gradient descent. In *Advances in Neural Information Processing Systems*, volume 33, pp. 19920–19930, 2020.
- Lucia Quirke, Louis Jaburi, David Johnston, William Z. Li, Gonçalo Paulo, Guillaume Martres, Girish Gupta, Stella Biderman, and Nora Belrose. Bergson: An open source library for data attribution. *arXiv preprint arXiv:2606.11660*, 2026.
- Andrea Schioppa, Polina Zablotskaia, David Vilar, and Artem Sokolov. Scaling up influence functions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 8179–8186, 2022. doi: 10.1609/aaai.v36i8.20791.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. [https://github.com/tatsu-lab/stanford\\_alpaca](https://github.com/tatsu-lab/stanford_alpaca), 2023.
- TransferLab Team. pyDVL: The python data valuation library. <https://pydvl.org/stable/>, 2024. Version 0.9.2.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers reinforcement learning, 2020. URL <https://github.com/huggingface/trl>.
- Jiachen T. Wang, Prateek Mittal, Dawn Song, and Ruoxi Jia. Data Shapley in one training run. In *International Conference on Learning Representations*, volume 2025, pp. 12358–12395, 2025a.
- Jiachen T. Wang, Dawn Song, James Zou, Prateek Mittal, and Ruoxi Jia. Capturing the temporal dependence of training data influence. In *International Conference on Learning Representations*, volume 2025, pp. 11236–11273, 2025b.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45. Association for Computational Linguistics, 2020.

Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 54104–54132. PMLR, 2024.

Chih-Kuan Yeh, Joon Kim, Ian En-Hsu Yen, and Pradeep Ravikumar. Representer point selection for explaining deep neural networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12): 3848–3860, 2023. doi: 10.14778/3611540.3611569.

## A DESIGN DETAILS

This appendix expands the design of §3: the canonical factorization for non-linear layer types (§A.1), the dimension-reduction maps and how they are applied to the factors (§A.2), optimizer-preconditioned capture (§A.3), the routing cost model (§A.4), the capture mechanism (§A.5), and the storage and streaming interfaces that connect capture to attribution (§A.6).

### A.1 A CANONICAL FACTORIZATION FOR ALL SUPPORTED LAYERS

Equation equation 1 is not specific to linear layers. For every supported layer type, `datttri-LLM` provides an *unfolding map* that reshapes the layer’s parameters into a matrix  $W \in \mathbb{R}^{N_o \times N_i}$  and rewrites its per-example gradient in the same form, with  $a_t$  an unfolded input vector,  $g_t$  the corresponding output gradient, and the sum running over  $T'$  unfolded positions in place of  $T$ . Convolutional layers unfold input patches (`im2col`); transposed convolutions use the same unfolding with the roles of input and output channels exchanged; embedding layers treat the one-hot token id as the input factor, so that materialization scatters  $g_t$  into the row of the token; and normalization layers (`LayerNorm`, `RMSNorm`, `GroupNorm`, `InstanceNorm`) use the normalized activation as the input factor, with the bias folded into an extra ones column where present. The supported types are `nn.Linear` (including Hugging Face `Conv1D`), `nn.Conv1d/2d/3d`, `nn.ConvTranspose1d/2d/3d`, `nn.Embedding` and `nn.EmbeddingBag`, and the normalization modules above. Because every operation in the library depends on a layer only through Eq. equation 1, a single implementation of each operation, and hence of each attribution method, covers all of them. Trainable parameters outside these families can still be captured through a batch-level parameter-gradient hook, which yields the materialized form directly.

### A.2 DIMENSION REDUCTION

§3.2 introduces factor projection (Eq. equation 2) and materialized projection. This section records how each is applied and which random maps are available.

**Factor projection.** The projection matrices  $P_a$  and  $P_g$  are applied inside the backward hook, so the raw factors are never buffered and only the projected pair  $(\tilde{a}_t, \tilde{g}_t)$  is retained. Because the result has the form of Eq. equation 1 with  $(N_i, N_o)$  replaced by  $(k_a, k_g)$ , every operation defined on factorized gradients, including routing (§A.4) and the Kronecker-factored statistics, applies to the projected factors without change. When  $k_a = k_g$  the construction coincides with the convention of

LoGRA (Choe et al., 2025) and GraSS (Hu et al., 2025). The two sides use distinct seeds derived from a per-layer base seed, which must be held fixed across every gradient that will be compared, including gradients captured at different steps or on different ranks.

**Materialized projection.** A single matrix  $P \in \mathbb{R}^{k \times N_i N_o}$  is applied to the flattened gradient,  $\tilde{\gamma} = P \text{vec}(DW)$ , as in TRAK (Park et al., 2023). The product is evaluated from the factors, since  $P \text{vec}(DW) = \sum_t P(a_t \otimes g_t)$ , and the result contracts the token axis into a single vector, so the choice of representation no longer arises and downstream comparisons reduce to dense inner products. Materialized projection is the only option for normalization layers, whose gradient is not an outer product of two factors, and it is incompatible with the Kronecker-factored methods, which require the factors.

**Random maps.** Either projection may use a dense Gaussian map (Johnson & Lindenstrauss, 1984), a Rademacher map with entries  $\pm 1/\sqrt{k}$  (Achlioptas, 2003), a sparse Johnson–Lindenstrauss transform with a fixed number of nonzeros per column (Kane & Nelson, 2014), or the sparsify-then-project map of GraSS (Hu et al., 2025). In addition, a *random mask* keeps a fixed random subset  $\mathcal{M}$  of the entries of  $DW$ . Because entry  $(i, j)$  of  $DW$  equals  $\sum_t g_t[i] a_t[j]$ , the masked entries are gathered directly from the factors as a sum of  $|\mathcal{M}|$  products per token position, without forming  $DW$ . The mask is the one reduction that preserves the coordinate identity of each retained entry, which is what optimizer preconditioning (§A.3) requires. All maps are generated from the same per-layer seed so that projected gradients from different steps, ranks, and passes remain comparable.

### A.3 OPTIMIZER-PRECONDITIONED CAPTURE

Modern models are trained with AdamW rather than SGD, and a growing family of attribution methods scores the *update* a training example induces rather than its raw gradient: LESS (Xia et al., 2024) compares the Adam direction of an example with a query gradient, and AdamW-influence (Deng et al., 2026) unrolls that direction through the remaining AdamW steps. For Adam-family optimizers the per-example update is the coordinate-wise function of Eq. equation 3, where  $(m, v)$  are the moments the coming `optimizer.step()` will update from. Eq. equation 3 has properties that shape the design. It needs the optimizer state *at the step the example was used*, so it must be evaluated during training; and it is nonlinear in  $g_z$  coordinate by coordinate, so it does not commute with a random projection, which mixes coordinates.

`dattri-LLM` directly applies optimizer preconditioning to the gradient factors. `HookManager` accepts the training optimizer, and for each hooked layer the per-example gradient entries are recovered from the factors and mapped through the optimizer’s update rule before they are buffered, so only the preconditioned copy ever exists, at the same place and with the same cost profile as factor projection. The random mask of §A.2 pairs naturally with it: the mask selects the coordinates, the entries are recovered from the factors, and the rule is applied to exactly those entries. Every coordinate-wise optimizer in `torch.optim` is supported through one kernel, and the map is switchable per pass so that a raw query pass can share the hooks of a preconditioned training pass. For trajectory methods a callback additionally records the moments on the same coordinates before and after every update, from which the batch gradient the optimizer consumed is recovered exactly. On this capture path, LESS is TracIn over preconditioned gradients with a cosine metric, in either its per-step or its checkpoint form, and AdamW-influence is a backward recurrence over the stored steps. The recurrence is linear in its summary matrix, so it can be carried either on the training side, at  $O(|S|^2)$  memory in the mask size  $|S|$ , or on the query side through the query gradients, which removes the mask entirely at a cost linear in the number of queries (§4.2).

### A.4 FLOP-AWARE ROUTING

Consider the cross-Gram matrix of gradient inner products between two batches of sizes  $B_1$  and  $B_2$ , with  $T$  tokens per example and layer dimensions  $N_i, N_o$ . Table 2 summarizes the factorized and materialized routes.

**Regimes.** The materialized route pays a one-time construction cost of  $TN_iN_o$  operations per example, amortized across all  $B_1B_2$  pairwise comparisons of cost  $N_iN_o$  each; the factorized route

Table 2: Complexity of computing a cross-Gram matrix between two batches of per-example gradients. Space denotes the storage required for the gradient representations and excludes the  $O(B_1 B_2)$  output matrix common to both routes. Under factor projection (Eq. equation 2),  $N_i$  and  $N_o$  are replaced by the projection dimensions  $k_a$  and  $k_g$ .

| Route        | Arithmetic operations                        | Representation space           |
|--------------|----------------------------------------------|--------------------------------|
| Factorized   | $O(B_1 B_2 T^2 (N_i + N_o))$                 | $O((B_1 + B_2) T (N_i + N_o))$ |
| Materialized | $O((B_1 + B_2) T N_i N_o + B_1 B_2 N_i N_o)$ | $O((B_1 + B_2) N_i N_o)$       |

avoids construction but pays  $T^2(N_i + N_o)$  per pair. Short sequences and wide layers therefore favor the factorized route, and long sequences or large comparison batches favor materialization. The storage comparison has a closed form: the factorized form is smaller exactly when  $T < N_i N_o / (N_i + N_o)$ . For unprojected wide layers this threshold is on the order of the layer width, so factors are kept; under factor projection it shrinks to  $k_a k_g / (k_a + k_g)$ , which realistic sequence lengths far exceed, so projected factors are better contracted into their (now small) outer products. The projected and unprojected regimes thus select opposite representations under the same rule.

**Decision.** Routing is evaluated per layer and per operation, for cross-Gram matrices and for per-example norms alike, by comparing the operation counts above and taking the cheaper route; the same routed kernels serve every attribution method and the data-selection callback. At capture time the representation that is persisted follows the storage rule and is exposed as a per-layer setting, with factor projection retaining factors and materialized projection contracting them. Because the routes are algebraically equivalent, routing does not change attribution scores in exact arithmetic. Appendix B.4 analyzes the case of reused query representations and evaluates routing across sequence lengths.

## A.5 CAPTURE MECHANISM

**Hook families.** `HookManager` attaches a hook family to each selected layer. The default *factorized* family registers a forward hook that records the layer input and a full backward hook that records the output gradient. A *parameter-gradient* family registers a tensor hook on each parameter and records the batch-level gradient in materialized form; it is the fallback for trainable layers outside the supported types. Layers are selected by type, by regular expression, or by an all-eligible marker, and layers whose type is not recognized automatically, such as custom normalization modules, are declared through per-layer overrides rather than by editing the model. When the training call is buried inside a framework, the capture context can be opened and closed from a framework callback instead of a `with` block.

**Loop-independent step detection.** Because the manager never sees the training loop, it infers when a step has finished from the hooks themselves. It tracks which hooked layers participated in the current forward pass and waits until every one has fired its backward hook, with an additional end-of-backward barrier for parameter gradients; layers skipped by the current control path are not waited for. Repeated invocations of a layer within a step (weight tying, multiple forward calls) are tracked as separate virtual layers. Under gradient checkpointing, the recomputed forward pass is recognized by the presence of a live autograd task and matched to the correct backward pass rather than double-counted. For backward passes that must not contribute, such as a validation pass used to form a target gradient, the manager can save, clear, and restore its per-step state around the extra pass, so the training step is captured exactly as if the extra pass had not occurred.

**Parallelism.** Under DDP (Li et al., 2020), parameter hooks fire after the all-reduce; under FSDP (Zhao et al., 2023), per-parameter hooks do not fire at all, so step completion is signaled through an end-of-backward callback queued on the autograd engine. In both cases each rank hooks its local shard and writes records independently, and the storage manager merges the per-rank indexes at read time, so sharded capture reproduces single-device gradients. The data-selection callback’s edit of `param.grad` is collective: contributions are all-reduced and applied to the full tensor under DDP or to the owned shard under FSDP, and under DDP the edit is queued to run after the reducer’s own write-back.

Table 3: Qwen checkpoints used in the scaling experiments: parameters (B), transformer blocks  $L$ , model width  $d$ , MLP width  $d_{\text{ff}}$ , attention heads (query / key-value), and hooked linear layers ( $7L$ ).

| Scale | Checkpoint   | Params | $L$ | $d$  | $d_{\text{ff}}$ | Heads | Hooked |
|-------|--------------|--------|-----|------|-----------------|-------|--------|
| 0.5B  | Qwen2.5-0.5B | 0.49   | 24  | 896  | 4864            | 14/2  | 168    |
| 1.5B  | Qwen2.5-1.5B | 1.54   | 28  | 1536 | 8960            | 12/2  | 196    |
| 3B    | Qwen2.5-3B   | 3.09   | 36  | 2048 | 11008           | 16/2  | 252    |
| 7B    | Qwen2.5-7B   | 7.62   | 28  | 3584 | 18944           | 28/4  | 196    |
| 14B   | Qwen2.5-14B  | 14.8   | 48  | 5120 | 13824           | 40/8  | 336    |
| 32B   | Qwen2.5-32B  | 32.5   | 64  | 5120 | 27648           | 40/8  | 448    |
| 72B   | Qwen2.5-72B  | 72.7   | 80  | 8192 | 29568           | 64/8  | 560    |
| 110B  | Qwen1.5-110B | 111    | 80  | 8192 | 49152           | 64/8  | 560    |

## A.6 STORAGE, IDENTITY, AND THE CAPTURE-ATTRIBUTION INTERFACE

**Sample identity.** Offloaded gradients are keyed by a content hash of a sample’s model inputs, which is independent of position and shuffling. The on-disk index maps `input_hash`  $\rightarrow$  (`step`, `sample_idx`)  $\rightarrow$  file, so retrieving one sample’s gradient is a direct slice rather than a scan of the batch record it was written in.

**Store.** Records are written before they are indexed and the index is persisted atomically, so an interrupted run leaves the store readable up to the last indexed record rather than corrupt. Under distributed training, each rank writes to its own subdirectory and a freshly constructed manager merges the per-rank indexes, so the number of ranks used for capture is invisible to the reader. The offload callback can merge the micro-batches of a gradient-accumulation window into one record stamped with the optimizer step, which is exact because parameters do not change within a window.

**Streaming interface.** Attributors consume gradients through one contract: a source yielding per-step (`step`, `gradient`, `hashes`) blocks. A disk-backed source re-iterates over pre-collected gradients, loading one file per item so a standard data loader can prefetch whole blocks. A live source computes blocks from a forward and backward pass, either as a re-iterable probe at a fixed checkpoint or, in its updating mode, advancing the optimizer so each step is a genuine point on a training trajectory; the updating mode mirrors the Hugging Face `Trainer`’s inner loop (optimizer and scheduler construction, mixed precision, clipping, and distributed wrapping). Attributors branch on exactly one property of the source, whether it is re-iterable, which methods with a pre-pass such as K-FAC’s Fisher fit require. Scores are returned as an `AttributionScore` whose rows are (example, step) pairs, so trajectory-aware and trajectory-agnostic methods share one container.

**Workflows.** The same attributor code serves the *on-the-fly* workflow, where an attributor drives a live source over the model and data and scores in one run, and the *store-then-attribute* workflow, where an ordinary training run caches gradients via the offload callback and attribution is performed later over the store, without the model and without another backward pass, so different methods and settings can be re-run over the same cache.

## B EXPERIMENT DETAILS

### B.1 THROUGHPUT UNDER A FIXED HARDWARE BUDGET

We evaluate the Qwen checkpoints in Table 3 on WikiText-103 with sequence length 512 and bf16 precision. Each configuration scores 512 training sequences against one query through 32B and four queries at 72B and 110B. The query count is matched across libraries at each scale. Each run receives four H200 GPUs (141 GB per device), 32 CPU cores, and 256 GB of host memory. Attention and MLP linear layers are included in attribution; embeddings and the language-model head are excluded.

**Library configurations.** `dattri-LLM`, Bergson and Kronfluence use FSDP. LogIX uses one model replica per device under DDP. Factor projection with  $k_a = k_g = 64$  is requested where

Table 4: Throughput (training sequences/s) on four H200s, with the per-device batch size in parentheses. The workload contains 512 training sequences and one query through 32B, and four queries at 72B and 110B. A dash denotes a configuration reported as out of memory.

| Method  | Library     | 0.5B       | 1.5B      | 3B        | 7B        | 14B       | 32B      | 72B     | 110B    |
|---------|-------------|------------|-----------|-----------|-----------|-----------|----------|---------|---------|
| GRADDOT | datttri-LLM | 610 (128)  | 299 (64)  | 186 (32)  | 110 (32)  | 50.0 (16) | 22.0 (8) | 8.6 (8) | 6.9 (4) |
|         | Bergson     | 68.8 (64)  | 57.9 (64) | 43.2 (32) | 39.8 (32) | 24.6 (16) | 14.3 (8) | 6.1 (8) | 4.9 (4) |
|         | Kronfluence | 177 (64)   | 111 (32)  | 64.4 (16) | 41.5 (16) | —         | —        | —       | —       |
|         | LogIX       | 65.8 (128) | 46.5 (64) | 34.8 (32) | 34.7 (32) | 17.2 (16) | 10.4 (8) | —       | —       |
| K-FAC   | datttri-LLM | 374 (128)  | 174 (64)  | 120 (32)  | 92.3 (32) | 40.3 (16) | 16.8 (8) | 7.4 (8) | 5.9 (4) |
|         | Bergson     | 2.8 (32)   | 1.9 (32)  | 1.2 (16)  | 0.8 (8)   | —         | —        | —       | —       |
|         | Kronfluence | 9.9 (64)   | 2.8 (32)  | 1.3 (16)  | —         | —         | —        | —       | —       |
|         | LogIX       | 61 (128)   | 44.8 (64) | 31.5 (32) | 32.6 (32) | 16.4 (16) | 9.7 (8)  | —       | —       |
| EK-FAC  | datttri-LLM | 337 (128)  | 174 (64)  | 116 (32)  | 83.9 (32) | 38.6 (16) | 16.1 (8) | 7.2 (8) | 5.6 (4) |
|         | Bergson     | 2.9 (32)   | 1.7 (16)  | 1 (8)     | 0.5 (4)   | —         | —        | —       | —       |
|         | Kronfluence | 8.3 (64)   | 2.3 (32)  | 1 (8)     | —         | —         | —        | —       | —       |
|         | LogIX       | 49.9 (128) | 37.5 (64) | 25.6 (32) | 24.2 (32) | 12.2 (16) | 6.9 (8)  | —       | —       |

Table 5: Peak per-device GPU memory (GB) at the batch sizes used in Table 4. Batch sizes vary across methods, libraries, and scales; these values document the throughput configurations and do not constitute a fixed-batch memory comparison.

| Method  | Library     | 0.5B  | 1.5B  | 3B    | 7B    | 14B  | 32B   | 72B   | 110B  |
|---------|-------------|-------|-------|-------|-------|------|-------|-------|-------|
| GRADDOT | datttri-LLM | 117.6 | 93.0  | 67.2  | 88.4  | 72.0 | 79.0  | 131.0 | 127.8 |
|         | Bergson     | 80.9  | 119.0 | 86.5  | 104.6 | 87.7 | 91.2  | 139.8 | 135.0 |
|         | Kronfluence | 101.7 | 95.3  | 80.7  | 119.7 | —    | —     | —     | —     |
|         | LogIX       | 117.4 | 94.6  | 70.9  | 97.4  | 91.0 | 124.4 | —     | —     |
| K-FAC   | datttri-LLM | 117.6 | 93.0  | 67.2  | 88.4  | 72.0 | 80.0  | 132.1 | 127.8 |
|         | Bergson     | 95.4  | 134.4 | 112.4 | 132.5 | —    | —     | —     | —     |
|         | Kronfluence | 101.7 | 95.3  | 98.6  | —     | —    | —     | —     | —     |
|         | LogIX       | 117.4 | 94.6  | 70.9  | 96.8  | 91.0 | 124.4 | —     | —     |
| EK-FAC  | datttri-LLM | 117.6 | 93.0  | 67.2  | 88.4  | 72.0 | 80.0  | 132.1 | 127.8 |
|         | Bergson     | 132.5 | 121.2 | 109.2 | 134.1 | —    | —     | —     | —     |
|         | Kronfluence | 108.7 | 122.6 | 100.4 | —     | —    | —     | —     | —     |
|         | LogIX       | 117.4 | 94.3  | 71.4  | 97.4  | 91.0 | 124.4 | —     | —     |

supported. Kronfluence runs without that projection. Consequently, the throughput comparison reflects the capabilities of the evaluated implementations as well as their execution efficiency. For each library, method, and model size, we manually test power-of-two per-device batch sizes and select the largest that can successfully complete the workload.

**Timing.** After two untimed warm-up steps, the GPU-synchronized timed interval covers the complete attribution computation: gradient capture, Kronecker-factor estimation and eigendecomposition where required, and scoring. Model loading, data preparation, and process initialization are excluded. For Bergson, which launches workers for successive pipeline stages, the reported time further removes the measured worker initialization and model-loading intervals for fairness. Throughput is 512 divided by the resulting elapsed time; it measures the complete finite workload and includes necessary per-call overheads for each attribution method.

**Memory measurements.** Table 5 reports peak per-device memory at the selected batch size. For datttri-LLM, LogIX, and Kronfluence we use `torch.cuda.max_memory_allocated`; Bergson uses NVML readings because its computation runs in child processes. The memory comparison at a common batch size is further reported below.

Table 6: GPU memory usage (GB) at batch size one, with 64 training sequences and one query. <sup>F</sup> denotes a completed sharded run on four H200s, with memory reported as an estimated aggregate across all four devices. All other completed runs use one H200. A dash indicates an out-of-memory failure under the evaluation protocol.

| Method  | Scale | dattri-LLM       | Bergson          | LogIX | Kronfluence |
|---------|-------|------------------|------------------|-------|-------------|
| GRADDOT | 0.5B  | 2                | 2.1              | 2.1   | 3.7         |
|         | 1.5B  | 4.7              | 4.8              | 4.9   | 10.2        |
|         | 3B    | 8.4              | 8.5              | 8.2   | 19.9        |
|         | 7B    | 17.8             | 17.3             | 17.8  | 43.5        |
|         | 14B   | 33.3             | 32.2             | 33.2  | 85          |
|         | 32B   | 70.6             | 69.9             | 71.8  | —           |
|         | 72B   | 323 <sup>F</sup> | 258 <sup>F</sup> | —     | —           |
|         | 110B  | 476 <sup>F</sup> | 354 <sup>F</sup> | —     | —           |
| K-FAC   | 0.5B  | 2.4              | 12               | 2     | 8.9         |
|         | 1.5B  | 6.7              | 45               | 4.9   | 31.7        |
|         | 3B    | 12.9             | 90.8             | 8.6   | 61.8        |
|         | 7B    | 28.5             | 440 <sup>F</sup> | 17.8  | —           |
|         | 14B   | 55.4             | —                | 33.2  | —           |
|         | 32B   | 122.2            | —                | 69.8  | —           |
|         | 72B   | 324 <sup>F</sup> | —                | —     | —           |
|         | 110B  | 476 <sup>F</sup> | —                | —     | —           |
| EK-FAC  | 0.5B  | 2.4              | 12.7             | 2     | 10.8        |
|         | 1.5B  | 6.7              | 45.1             | 4.6   | 37.6        |
|         | 3B    | 12.9             | 90.8             | 8.6   | 73.7        |
|         | 7B    | 28.5             | 440 <sup>F</sup> | 17.8  | —           |
|         | 14B   | 55.4             | —                | 33.2  | —           |
|         | 32B   | 122.2            | —                | 70.7  | —           |
|         | 72B   | 324 <sup>F</sup> | —                | —     | —           |
|         | 110B  | 476 <sup>F</sup> | —                | —     | —           |

**Memory Scaling at Batch Size One** We evaluate GPU memory usage at batch size one to assess the largest model each library can accommodate, using the same models and workload as above. Each configuration is evaluated on one H200; when a single device is insufficient, we use four H200s if the library supports sharding. LogIX does not support sharding in its current implementation. Table 6 explicitly marks completed four-device sharded runs and reports their estimated aggregate memory usage across all four devices. A dash indicates that the configuration runs out of memory under this evaluation protocol.

## B.2 FIDELITY EVALUATION DETAILS

We evaluate three models from three families: GPT-2 (124M), Qwen2.5-1.5B, and OLMo-3-7B (Groeneveld et al., 2024). For each model, all parameters are trained for one epoch on 512 blocks of 128 WikiText-2 tokens, drawn with the model’s own tokenizer, using batch size 32 for 16 AdamW updates. The optimizer uses  $\beta = (0.9, 0.999)$ ,  $\epsilon = 10^{-8}$ , and zero weight decay. The learning rate peaks at  $10^{-5}$  after linear warm-up over the first 10% of steps, followed by linear decay. GPT-2 and Qwen2.5 are trained by the evaluation’s own training loop. The OLMo models are trained by OLMo-core, the trainer of the OLMo releases, without modification: the released weights are converted once into an OLMo-core checkpoint, the learning-rate schedule is passed as a scheduler configuration, and dattri-LLM captures gradients by wrapping the trainer’s `fit()` call, on one GPU. For TSLOO (Wang et al., 2025b), each of 50 randomly selected training blocks is removed from its original batch in a separate replay of training. We record the change in loss on 64 held-out validation blocks. Fidelity is the Spearman correlation between attribution scores and these loss changes over the 50 training blocks, computed separately for each validation block and averaged over the 64 blocks. For numerical precision, we run training and evaluation use fp32, with TF32 disabled, eager attention, deterministic kernels, and dropout disabled. Pretrained weights are loaded once, and the reference run and all 50 removal runs start from the same copy. Each model scale uses a fixed seed, and we average over the the 64 query correlations to get robust results.

Table 7: Resource limits in the fidelity benchmark under the budget of one B200, 256 GB of host memory, and 1 TB of local disk.

| Method                                            | Model        | Limiting resource |
|---------------------------------------------------|--------------|-------------------|
| MAGIC (Bergson)                                   | Qwen2.5-1.5B | GPU memory        |
| AdamW-influence (full, <code>datttri-LLM</code> ) | Qwen2.5-1.5B | GPU memory        |
| MAGIC                                             | OLMo-3-7B    | GPU memory        |
| AdamW-influence (full)                            | OLMo-3-7B    | GPU memory        |
| SOURCE (Bergson)                                  | OLMo-3-7B    | Local disk        |
| EK-FAC (Bergson)                                  | OLMo-3-7B    | GPU memory        |

**Methods.** `datttri-LLM` implements AdamW-influence with full-dimensional gradients and with random coordinate masks. The reduced configuration uses ten disjoint masks of 512 coordinates per layer, computes scores independently for each mask, and averages them. Its measured cost includes all ten masks. We also evaluate full-dimensional EK-FAC and EK-FAC with factor projection  $k_a = k_g = 64$ . From Bergson (v0.26.1), we evaluate MAGIC (Ilyas & Engstrom, 2025), SOURCE (Bae et al., 2024), TrackStar (Chang et al., 2025), and EK-FAC (George et al., 2018). All EK-FAC implementations use damping equal to 0.1 times the mean eigenvalue of each layer. TrackStar uses per-module projection dimension 64. SOURCE uses four segments of four steps with two checkpoints per segment. MAGIC uses Bergson’s functional trainer to replay the trajectory and differentiate each query loss through the training run; its replay agrees with the reference within  $10^{-5}$  in validation loss. On OLMo-3-7B the Bergson method that scores a trained model (TrackStar) takes it through its Hugging Face export; MAGIC and SOURCE own the training loop and cannot run under OLMo-core’s trainer. Every method scores the same 64 queries against the same training trajectory.

**Batch sizes.** Every method captures the training gradients in batches of 8 blocks. The full-dimension methods score the 64 queries in chunks, and the chunk is, for each library, the largest power of two that completes within the budget: 64, 8, and 8 queries for `datttri-LLM`’s EK-FAC on GPT-2, Qwen2.5-1.5B, and OLMo-3-7B, and 16, 1, and  $N/A$  for Bergson’s EK-FAC and SOURCE. At 7B, `datttri-LLM`’s EK-FAC keeps its Kronecker factors in host memory and moves one layer’s factors to the GPU at a time.

**Resource budget.** Each execution receives one B200 (183 GB), 16 CPU cores, 256 GB of host memory, and 1 TB of local disk; the 7B model’s fp32 training state (16 bytes per parameter for weights, gradients, and the two Adam moments) does not fit a smaller GPU. The reported time is the attribution of all 64 queries: training the trajectory, loading the model and the data, and the TSLOO reference construction are excluded, since a model is trained whether or not it is attributed. For the single-checkpoint methods the time runs from the trained model to the scores; for the trajectory-based methods it covers the attribution pass over the recorded trajectory, which for MAGIC is the backward pass through training. Throughput in Figure 3 is 64 queries divided by this time. Table 7 further records the limiting resource.

### B.3 FIXED-BATCH COMPARISONS AT A FIXED MODEL SIZE

In addition to the maximum-throughput analysis in the main text, we further evaluate all libraries at the same batch size to examine performance differences under matched conditions. We compare `datttri-LLM`, LogIX, Bergson, and Kronfluence on Pythia-0.5B (Biderman et al., 2023), attributing language-modeling loss on 1024 WikiText-103 training sequences of length 512, with either one or 16 queries. Completed configurations use training batch size 8 on one A40 (46 GB) in fp32. We evaluate both factor projection with  $k_a = k_g = 64$  and full-dimensional gradients; projection uses a capture-time Rademacher map where configurable. Table 9 reports runtime and memory usage for both query counts. Note that these supplementary experiments use a different model and hardware configuration from the Qwen throughput benchmark.

**Results.** `datttri-LLM` has the lowest runtime in nine of the twelve rows of Table 9, and in every row that uses a curvature-based method. For projected K-FAC and EK-FAC it runs 1.6–2.5 $\times$  faster

Table 8: Complete fidelity results on one B200: Spearman correlation with TSLOO ( $\rho$ ), attribution time of the 64 queries in seconds ( $t$ ), and peak GPU memory in GB. “-”: infeasible under the budget.

| Library     | Method                  | GPT-2  |      |    | Qwen2.5-1.5B |       |    | OLMo-3-7B |      |     |
|-------------|-------------------------|--------|------|----|--------------|-------|----|-----------|------|-----|
|             |                         | $\rho$ | $t$  | GB | $\rho$       | $t$   | GB | $\rho$    | $t$  | GB  |
| datttri-LLM | AdamW-inf. ( $k=512$ )  | .680   | 10   | 8  | .526         | 51    | 47 | .437      | 73   | 136 |
| datttri-LLM | AdamW-inf. ( $k=8192$ ) | .762   | 28   | 8  | .639         | 103   | 49 | .633      | 170  | 138 |
| datttri-LLM | AdamW-inf. (full)       | .845   | 1137 | 25 | -            | -     | -  | -         | -    | -   |
| datttri-LLM | EK-FAC ( $k=64$ )       | .699   | 15   | 8  | .420         | 50    | 47 | .303      | 126  | 40  |
| datttri-LLM | EK-FAC (full)           | .664   | 17   | 30 | .434         | 447   | 96 | .402      | 3444 | 139 |
| Bergson     | MAGIC                   | .833   | 1265 | 24 | -            | -     | -  | -         | -    | -   |
| Bergson     | SOURCE                  | .773   | 846  | 13 | .275         | 28026 | 64 | -         | -    | -   |
| Bergson     | TrackStar               | .523   | 34   | 8  | .332         | 289   | 47 | .374      | 368  | 56  |
| Bergson     | EK-FAC                  | .681   | 171  | 13 | .432         | 4560  | 64 | -         | -    | -   |

Table 9: **Cross-library efficiency under matched batch sizes.** Results use Pythia-0.5B on one A40 (46 GB), with 1024 WikiText-103 training sequences and either one or 16 queries.  $T$ : attribution wall-clock time (s), excluding model construction and data loading;  $M$ : peak GPU memory (GB) over the same phases. The lowest runtime in each row is bold. Completed configurations use training batch size 8. OOM indicates an out-of-memory failure at every tested batch size down to one; N/A indicates an unsupported combination.

|                                       |        | datttri-LLM |          | LOGIX    |          | BERGSON   |          | KRONFLUENCE |          |
|---------------------------------------|--------|-------------|----------|----------|----------|-----------|----------|-------------|----------|
| Method                                | Proj.  | <i>T</i>    | <i>M</i> | <i>T</i> | <i>M</i> | <i>T</i>  | <i>M</i> | <i>T</i>    | <i>M</i> |
| One query ( $n_{\text{test}} = 1$ )   |        |             |          |          |          |           |          |             |          |
| GRADDOT                               | $k=64$ | 41          | 7        | 68       | 7        | <b>39</b> | 8        | N/A         | N/A      |
|                                       | full   | <b>45</b>   | 11       | OOM      | —        | 53        | 13       | 50          | 11       |
| K-FAC                                 | $k=64$ | <b>54</b>   | 10       | 89       | 7        | 180       | 20       | N/A         | N/A      |
|                                       | full   | <b>134</b>  | 19       | OOM      | —        | 194       | 20       | 201         | 12       |
| EK-FAC                                | $k=64$ | <b>58</b>   | 10       | 143      | 7        | N/A       | N/A      | N/A         | N/A      |
|                                       | full   | <b>223</b>  | 21       | OOM      | —        | 278       | 27       | 347         | 15       |
| 16 queries ( $n_{\text{test}} = 16$ ) |        |             |          |          |          |           |          |             |          |
| GRADDOT                               | $k=64$ | 40          | 13       | 71       | 7        | <b>39</b> | 8        | N/A         | N/A      |
|                                       | full   | 58          | 31       | OOM      | —        | 121       | 30       | <b>57</b>   | 28       |
| K-FAC                                 | $k=64$ | <b>55</b>   | 18       | 90       | 7        | 225       | 44       | N/A         | N/A      |
|                                       | full   | <b>153</b>  | 34       | OOM      | —        | 317       | 44       | 210         | 28       |
| EK-FAC                                | $k=64$ | <b>59</b>   | 18       | 146      | 7        | N/A       | N/A      | N/A         | N/A      |
|                                       | full   | <b>230</b>  | 38       | OOM      | —        | 461       | 44       | 360         | 28       |

than LogIX and  $3.3\text{--}4.1\times$  faster than Bergson; for the full-dimensional variants it runs  $1.2\text{--}1.6\times$  faster than Kronfluence and  $1.4\text{--}2.1\times$  faster than Bergson. The remaining three rows are GRADDOT, where the runtimes of datttri-LLM, Bergson, and Kronfluence differ by at most two seconds. At this scale a single forward and backward pass over the 1024 training sequences in fp32 accounts for most of the runtime, and GRADDOT adds little to it, so any library that runs the model once lands near the same floor; LogIX, which extracts gradients one query at a time through a disk-backed store, does not. The curvature-based methods add a fit over the training gradients and a per-layer preconditioning of the queries, and it is this attribution-side work that separates the libraries. Moving from one query to 16 leaves the runtime of datttri-LLM almost unchanged (+0–19 s), whereas Bergson’s full-dimensional K-FAC and EK-FAC take  $1.6\text{--}1.7\times$  longer.

#### B.4 ROUTING ACROSS SEQUENCE LENGTHS

In this section, we show the effectiveness of the heuristic proposed in §3.2 We use the full-dimensional GRADDOT setting of Appendix B.3: Pythia-0.5B, WikiText-103, fp32, one query, and

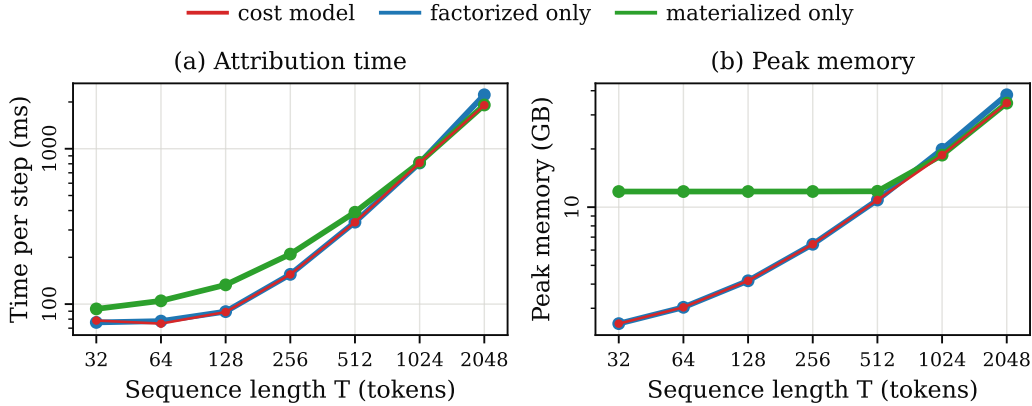


Figure 6: **Adaptive routing versus fixed gradient representations.** Mean attribution time per step (left) and peak GPU memory (right) for full-dimensional GRADDOT on Pythia-0.5B as sequence length increases, using batch size 8, one query, and fp32 on one A40. The cost-model heuristic selects between factorized and materialized execution; fixed-route ablations use the same implementation. Adaptive routing closely tracks the faster fixed configuration while maintaining the lowest or tied peak memory at every evaluated length. Memory includes all GPU allocations.

one A40. Sequence length ranges from 32 to 2048 tokens, and all configurations use batch size 8. We time 128 steps after 8 warm-up steps and report mean attribution time per step and peak GPU memory. The three `dattri-LLM` configurations differ only in their routing decision; the fixed-route runs are ablations of the same implementation. Figure 6 presents the curves.

**Adaptive versus fixed execution.** The maximum runtime excess over the faster fixed configuration is approximately 2.2%, computed from the rounded table entries. For  $32 \leq T \leq 512$ , materialized execution takes 13–43% longer than adaptive routing. At  $T = 2048$ , factorized execution takes approximately 16% longer. Noticeably, while the heuristic is developed for runtime optimization, the short-sequence memory advantage is also substantial: at  $T = 32$ , the always-materialized run has  $4.7\times$  the peak memory of adaptive execution. These are total GPU-memory measurements, not isolated gradient-buffer sizes. Adaptive routing has the lowest or tied memory among the three `dattri-LLM` configurations at every evaluated length.

## B.5 TOKEN-LEVEL ATTRIBUTION EXAMPLES

We illustrate token-level attribution using GRADDOT, K-FAC, and EK-FAC on Pythia-0.5B and WikiText-103. For these methods, the sequence-level score decomposes over the training sequence’s token positions. Writing the query-side matrix as  $R$ , the contribution of position  $t$  is  $s_t = g_t^\top R a_t$ , and the sum over positions recovers the sequence-level score. Here  $R$  is the query gradient for GRADDOT, its K-FAC preconditioned form for K-FAC, or its EK-FAC preconditioned form for EK-FAC. All attention and MLP linear layers are attributed without projection. K-FAC and EK-FAC fit curvature on 32 WikiText-103 paragraphs with damping  $10^{-3}$ . Each panel is normalized independently because the methods produce scores on different scales. The figures illustrate reuse of the captured factors for fine-grained analysis; they do not establish a token decomposition for every attribution method in the library.

## B.6 WEIGHT-GRADIENT-FREE CAPTURE FOR FROZEN ATTRIBUTION

Attribution at a fixed checkpoint reads only the inputs and output gradients of the hooked layers; the weight gradients that the backward pass also computes are never used. `dattri-LLM` therefore provides an optional capture mode (`invasive.linear.io`) that, during capture, routes each hooked `nn.Linear` through an operation whose backward pass returns only the input gradient. It enables more efficient exact attribution by eliminating the memory required for parameter gradients

**GradDot**

plain maskray generally hunts at the surface of the bottom substrate , rather than digging for prey . Its diet consists predominantly of caridean shrimp and polychaete worms . Small bony fishes are also eaten , along with the occasional penaeid prawn or amphipod . Larger rays consume a greater variety of prey and relatively more polychaete worms when compared to smaller rays . This species is parasitized by the tapeworm Acanthobothrium jonesi .

**K-FAC**

plain maskray generally hunts at the surface of the bottom substrate , rather than digging for prey . Its diet consists predominantly of caridean shrimp and polychaete worms . Small bony fishes are also eaten , along with the occasional penaeid prawn or amphipod . Larger rays consume a greater variety of prey and relatively more polychaete worms when compared to smaller rays . This species is parasitized by the tapeworm Acanthobothrium jonesi .

**EK-FAC**

plain maskray generally hunts at the surface of the bottom substrate , rather than digging for prey . Its diet consists predominantly of caridean shrimp and polychaete worms . Small bony fishes are also eaten , along with the occasional penaeid prawn or amphipod . Larger rays consume a greater variety of prey and relatively more polychaete worms when compared to smaller rays . This species is parasitized by the tapeworm Acanthobothrium jonesi .

Figure 7: **maskray** (train row 308). Target: “The plain maskray is a species of stingray that feeds on caridean → **shrimp**”. Under GRADDOT the largest positions are the subword *ts* of *hunts*, *mask*, and the two occurrences of *of*, with *shrimp* itself at rank 6. Both preconditioned methods concentrate on *of caridean*, the phrase the target completes.

Table 10: Speedup of weight-gradient-free capture over standard capture (standard time / weight-gradient-free time) for *datttri*-LLM on Pythia-0.5B, one L40S (48 GB), fp32, with the workload of Table 9. Each entry is the median over five paired repetitions.

| Method  | Projection | $n_{\text{test}} = 1$ | $n_{\text{test}} = 16$ |
|---------|------------|-----------------------|------------------------|
| GRADDOT | $k = 64$   | 1.19×                 | 1.18×                  |
| GRADDOT | full       | 1.16×                 | 1.09×                  |
| K-FAC   | $k = 64$   | 1.14×                 | 1.11×                  |
| K-FAC   | full       | 1.10×                 | 1.07×                  |
| EK-FAC  | $k = 64$   | 1.10×                 | 1.14×                  |
| EK-FAC  | full       | 1.08×                 | 1.08×                  |

and reducing the runtime overhead of gradient collection. Because the mode produces no parameter gradients, it is applicable only outside training; capture within a training loop, including online data selection, uses standard capture. Kronfluence and Bergson avoid the same computation by freezing the model parameters during attribution, whereas LogIX have to compute weight gradients for the layers it tracks. In Table 9, *datttri*-LLM uses this mode for GRADDOT and full-dimensional K-FAC and EK-FAC, and standard capture for projected K-FAC and EK-FAC. To quantify its effect, we rerun each *datttri*-LLM configuration of Table 9 under both modes, changing only the capture mode, on one L40S (48 GB) in fp32 with five paired repetitions. As Table 10 shows, standard capture takes 7–19% longer and uses at most 4.5 GB more peak memory, which is the cost of the weight-gradient computation it retains.

**GradDot**

attended Emsley A. Laney High School in Wilmington , where he anchored his athletic career by playing baseball , football , and basketball . He tried out for the varsity basketball team during his sophomore year , but at 5 ' 11 " ( 1.80 m ) , he was deemed too short to play at that level . His taller friend , Harvest Leroy Smith , was the only sophomore to make the team .

**K-FAC**

attended Emsley A. Laney High School in Wilmington , where he anchored his athletic career by playing baseball , football , and basketball . He tried out for the varsity basketball team during his sophomore year , but at 5 ' 11 " ( 1.80 m ) , he was deemed too short to play at that level . His taller friend , Harvest Leroy Smith , was the only sophomore to make the team .

**EK-FAC**

attended Emsley A. Laney High School in Wilmington , where he anchored his athletic career by playing baseball , football , and basketball . He tried out for the varsity basketball team during his sophomore year , but at 5 ' 11 " ( 1.80 m ) , he was deemed too short to play at that level . His taller friend , Harvest Leroy Smith , was the only sophomore to make the team .

Figure 8: **jordan** (train row 1724). Target: “Michael Jordan attended Emsley A. Laney High School in → **Wilmington**”. The paragraph states the fact. GRADDOT ranks *Wil* second, behind the function word *in*, and shades the whole paragraph; K-FAC and EK-FAC put *Wil* first, followed by *in* and the remaining subwords of the city name, and leave the text after the school name essentially blank.

## B.7 DATA SELECTION DETAILS

Training uses Llama-3.2-1B with AdamW, a linear learning-rate schedule with 3% warm-up, gradient clipping at 1.0, and batch size 8, over three seeds. The learning rate is  $10^{-6}$  for full-parameter fine-tuning and  $10^{-4}$  for LoRA. Master weights are stored in fp32, with bf16 autocast during training. The source pool is Alpaca and the target task is SAMSum, with the same chat template applied to both datasets. At each step, the scoring target is one example from a 16-example SAMSum validation pool. Selection removes the bottom half of each batch according to GRADDOT scores against that example. Full-parameter runs attribute all transformer attention and MLP linear layers, excluding embeddings and the language-model head. LoRA uses rank 8 and  $\alpha = 16$  on the same projections; capture and gradient intervention are restricted to the adapter layers. Test perplexity is evaluated every 50 steps on the response tokens of 500 held-out SAMSum examples, with prompt tokens masked.

**Runtime.** Figure 10 reports per-step runtime under full-parameter fine-tuning, averaged over steps after warm-up. Standard training takes 89 ms per step, selection with `datttri-LLM` takes 146 ms, and the dedicated implementation of Hu et al. (2026) takes 154 ms. The library implementation is approximately 5% faster than the dedicated implementation. The dedicated implementation uses an additional forward pass to obtain training gradients, whereas `datttri-LLM` captures the required quantities from the training backward pass. Scoring and selection account for the remaining application overhead.

**GradDot**

order to offset the pace of the group 's previous album , Slayer deliberately slowed down the album 's tempo . In contrast to their previous albums , the band utilized undistorted guitars and toned-down vocals . While some critics praised this musical change , others – more accustomed to the style of earlier releases – were disappointed . The songs " Mandatory Suicide " and the title track , however , have become permanent features of the band 's live setlist .

**K-FAC**

order to offset the pace of the group 's previous album , Slayer deliberately slowed down the album 's tempo . In contrast to their previous albums , the band utilized undistorted guitars and toned-down vocals . While some critics praised this musical change , others – more accustomed to the style of earlier releases – were disappointed . The songs " Mandatory Suicide " and the title track , however , have become permanent features of the band 's live setlist .

**EK-FAC**

order to offset the pace of the group 's previous album , Slayer deliberately slowed down the album 's tempo . In contrast to their previous albums , the band utilized undistorted guitars and toned-down vocals . While some critics praised this musical change , others – more accustomed to the style of earlier releases – were disappointed . The songs " Mandatory Suicide " and the title track , however , have become permanent features of the band 's live setlist .

Figure 9: **slayer** (train row 752). Target: “South of Heaven is a studio album by the American thrash metal band → **Slayer**” (tokenized `Slayer`). The answer’s first subword ranks first under all three attributors, but its lead over the runner-up grows from  $3.6\times$  (GRADDOT) to  $7.3\times$  (EK-FAC) and  $35.7\times$  (K-FAC). The paragraph discusses the album’s tempo rather than restating the band’s name, and the sequence-level GRADDOT score is in fact negative ( $-82$ ), whereas both preconditioned scores are positive: the preconditioner isolates the one position that supports the fact from the many that do not.

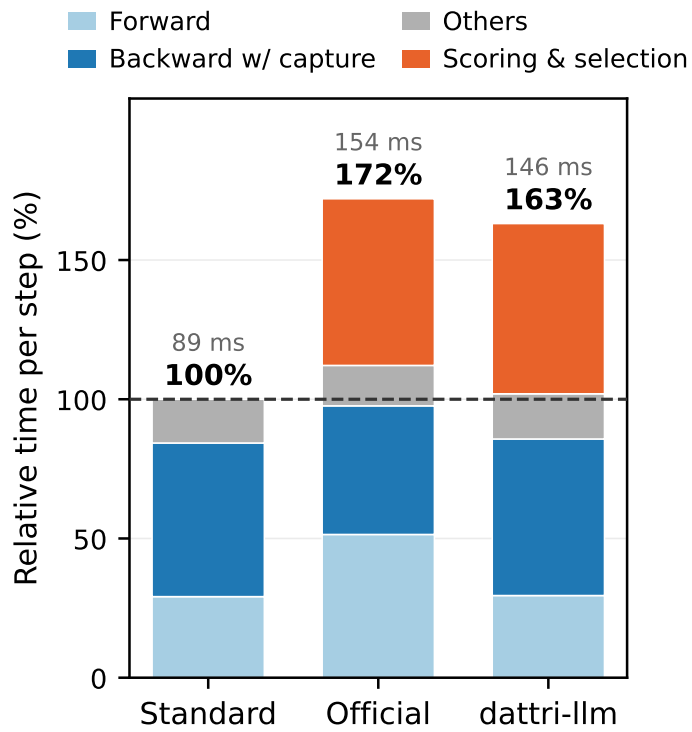


Figure 10: Per-step runtime of online data selection relative to standard training, under full-parameter fine-tuning.